

Macaulay2 Language

Arithmetic +, -, *, ^ do what you expect

/	division
//	integer division
%	modular remainder (≥ 0)
sqrt	square root
!	factorial
floor, ceiling	floor & ceil

Packages

- `needsPackage` loads a package if not already loaded
- `loadedPackages` lists loaded packages

Lists & Sequences

List	Sequence
<code>{1,2,"hi"}</code>	<code>(1,2,"hi")</code>
vector operations	no vector ops
<code>flatten</code>	<code>splice</code>

Both are immutable & zero-indexed. If `L` is a `List` or a `Sequence`, can...

- Use `#` to get items (via `L#0`) or length (via `#L`)
- Use `_` to get (multiple) items, via `L_0` or `L_{0,1,4}`
- `append(L,"last"), prepend("first", L), insert(2, "middle", L)`
- `drop(L,{2,2})` removes item at *index* 2; `delete(L,2)` removes all items with *value* 2

If `f` is a function, the following are variants on the idea of looping through & applying `f`

- `scan(L,f)` applies `f` to each element & discards return
- `apply(L,f)` returns list of `f` applied to each element

When `f` takes 2 args...

- `fold(f,L)` iteratively applies `f` to next elem & prev result. `fold(L,f)` does same but starts at end of list.
- `accumulate(f,L)` is like `fold` but returns list of intermediate results. `accumulate(L,f)` does same but starts at end of list.

If `g` is a `true/false` function, can get sub-list/sequences: `select(L,g)`, `positions(L,g)`, and to count, `number(L,g)`

Defining Functions

Example syntax	What it does
<code>f = x -> x^2</code>	$f(x) = x^2$
<code>g = y -> (i:=2; i*y)</code>	$g(y) = 2y$
<code>a = (r,s) -> r+s</code>	$a(r,s) = r + s$

- `;` separates statements
- Need `()` around body if multiple statements
- use `:=` instead of `=` inside (unless you *want* global variable)

Control Structures For B boolean expr, m,n ints...

- **if B then X else Y**
Eval B, if true eval & return X else do same for Y. If omit **else**, that branch evals to null
- **while B list X do Y**
Eval B, when true, eval X and save; eval Y and discard; repeat. At end, return list of X vals. Can omit **list X** or **do Y**
Ex: `i=0; while i<3 list i^2 do i=i+1` returns `{0,1,4}`
- **for i from m to n when B list X do Y**
Inits `i=m`, and as long as `i ≤ n`, continue. Eval B, if true continue. Eval X and save; eval Y and discard; repeat. At end, return list of X vals. Can omit **when p** and/or **list X** and/or **do Y**

Scripting

- `scriptCommandLine` is a list of command line arguments as strings (0 = file name)
- `value` evaluates a string as M2 code

Getting help

Command	Usage & Comments
<code>help</code>	use alone to get generic menu; use with function to get documentation for that function; combine with below commands
<code>methods</code>	<code>methods X</code> for X function, type, keyword, or package lists methods “associated” with X; <code>methods(X,Y)</code> for types X & Y gives methods involving both
<code>code</code>	use with (list of) functions to display code, or combine w/ <code>methods</code>
<code>about</code>	get (list of) relevant documentation bits for string, function, symbol, or type. Combine w/ <code>help</code>
<code>apropos</code>	list of global symbols matching string. Regex ok; case sensitive.

Math in M2

(Polynomial) Rings

Built-in “coefficient rings” are:

- Exact: ZZ, QQ, ZZ/p, GF(p^n) (for p prime)
- Inexact: RR, CC (use ii for i)

For coeff ring R , define 4-var polynomial rings via:

- $R[\text{alpha}, \text{beta}, \text{gamma}, \text{delta}]$
- $R[w..z]$ or $R[\text{vars}(22..25)]$
- $R[4]$ (gives subscripted vars)

Vars must be SYMBOLS, use **symbol** x if you’ve say already defined x to be something

Other rings/fields:

- Quotient rings: R/I for I ideal, or R/s for s sequence of ring elements
- Fraction field: $\text{frac } R$ for R domain
- If R is a field but M2 doesn’t realize, use $\text{toField } R$

Working with multiple rings

- use R makes R current ring
- for R a “basing” of S and $f \in S$, $\text{lift}(f, R)$ views f as element of R , if possible
- for R a “basing” of S and $g \in R$, $\text{promote}(g, S)$ views g as element of S
Ex: $\text{promote}(1/2, \text{RR})$ outputs 0.5

Miscellaneous

- Use **substitute** or **sub** to (partially) evaluate polynomials.
Ex: $\text{sub}(x*y+z, \{x=>2, z=>3\})$ gives $2y + 3$
- Use **gens** and **vars** to access generators, as list and matrix (respectively)

Ideals

To make ideal, **ideal**. Ideal operations:

$+, *, ^$	add, multiply, powers
isSubset	containment
$==, !=$	check equality
$:$	colon ideal

For f ring element, I ideal:

$f \% I$ reduce (“remainder mod I ”)
 $f // \text{gens } I$ decompose into combo of gens

Dimension/Length + Leading Terms

Set monomial order when defining ring, e.g., $\text{ZZ}/2[x,y, \text{MonomialOrder} \Rightarrow \text{Lex}]$.

Options are **Lex** = alphabetical; **GLex** = graded lex; **GRevLex** = default, fastest but least intuitive

Given an ideal I in ring R ...

- **first entries gens** $gb\ I$ returns Gröbner basis as list
- **leadTerm** I gives leading term ideal
- **leadTerm** $gb\ I$ puts leading terms in matrix
- **length**($R^1/\text{leadTerm } I$) gives dimension of R/I . If I mono. ideal, can skip **leadTerm**

Do NOT use **dim** command; will always give zero.

Maps: Use target before source!

- **map**(S, R) for rings gives “identity” map $R \rightarrow S$, i.e., tries to match up variable names & sends to zero if can’t
Ex: $R = \text{ZZ}[w,x,y], S = \text{ZZ}[x,y,z]$ then $\text{map}(S, R)$ is $w \mapsto 0, x \mapsto x, y \mapsto y$
- For d list of images (or list of options) **map**(S, R, d) is map defined by d
- Similar for other kinds of objects (modules, chain complexes)

Matrices: use double-nested list & **matrix**

Ex: $\text{matrix} \{\{1,2\}, \{3,4\}\}$

For matrix M ...

- Back into double-nested list via **entries** M .
- **flatten entries** M to get single list
- **first entries** M to get just 1st row

Modules

- To view ring or ideal as module, use **module**.
- Given matrix, use **ker**, **coker**, and **image**
- Create submodules & quotients using expected math notation

The TestIdeals package

- **frobenius**(e, I) gives ideal to p^e -th power.
Or just **frobenius**(I) for $e = 1$
- **isFPure**(I) checks if S/I is F-split;
isFPure(R) checks if R is F-split